

Architecture de la solution Matchia

La solution Matchia repose sur une architecture client-serveur en couches. Le frontend, développé avec React et TypeScript, constitue une application web autonome qui communique avec une API REST réalisée avec Spring Boot. Le backend centralise la sécurité, les règles métier et l'accès aux données. Cette séparation permet de faire évoluer l'interface et la logique applicative de manière indépendante, tout en conservant un contrat d'échange explicite fondé sur HTTP et JSON.

Choix architectural retenu. Matchia ne met pas en œuvre une architecture microservices : les capacités fonctionnelles sont regroupées dans une même application backend. Ce backend reste cependant organisé en modules métier cohérents, notamment l'administration SaaS, les marketplaces, les stores et modules, les produits, les abonnements, les partenariats, les contrats et les demandes de financement. La qualification la plus précise est donc celle d'une architecture monolithique modulaire en couches, avec un frontend séparé et une gestion multi-tenant logique.

Deux vues complémentaires sont nécessaires pour décrire cette organisation. La vue logique explique la répartition des responsabilités dans le code ; la vue physique montre les composants qui communiquent pendant l'exécution.

Vue	Question traitée	Éléments représentés
Logique	Comment le logiciel est-il organisé ?	Vue React, contrôleurs REST, services métier, modèle, repositories et persistance.
Physique	Quels composants communiquent à l'exécution ?	Navigateur, frontend, backend, PostgreSQL et services externes.

Architecture logique

L'architecture logique décrit l'enchaînement des responsabilités depuis l'interaction utilisateur jusqu'à la persistance. La figure 1 propose une lecture inspirée du modèle MVC. Cette correspondance est pédagogique plutôt que stricte : la vue n'est pas générée par le serveur, mais portée par une application React séparée, tandis que le backend expose des ressources REST et suit principalement l'organisation Controller-Service-Repository.

Architecture de Matchia

L'architecture de Matchia est principalement une architecture monolithique en couches, inspirée du modèle MVC, avec un frontend séparé.

Ce n'est pas une architecture microservices.

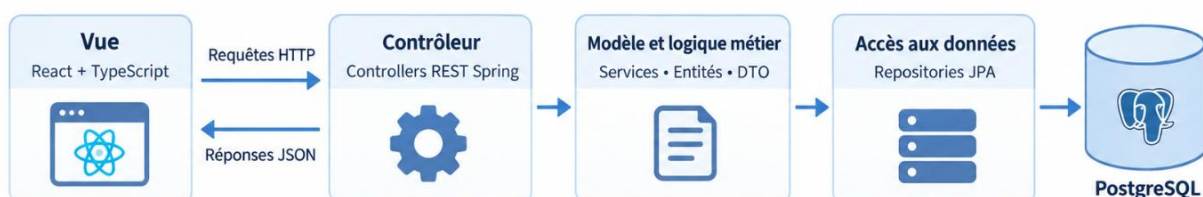


Figure 1 — Architecture logique de la plateforme Matchia

Couche de présentation

La couche de présentation est développée avec React et TypeScript. Elle regroupe les pages, les composants d'interface, les layouts, la navigation et les services HTTP utilisés par les espaces Public, Client, Administrateur Banque, Concessionnaire et Administrateur SaaS. Elle collecte les actions de l'utilisateur, applique les validations d'ergonomie immédiates et transmet les demandes au backend. Les décisions de sécurité et les règles métier sensibles ne reposent toutefois jamais uniquement sur le navigateur.

Couche API et contrôleurs

Les contrôleurs Spring Boot constituent le point d'entrée du backend. Ils exposent les endpoints REST, reçoivent les paramètres et les corps de requête, déclenchent la validation des données et délèguent le traitement aux services appropriés. Les échanges avec le frontend utilisent principalement le format JSON. Les objets de transfert de données, ou DTO, définissent le contrat de l'API et limitent les informations exposées ; ils appartiennent donc à la frontière applicative plutôt qu'au modèle de persistance lui-même.

Services métier et modèle

La couche métier porte les règles qui donnent son sens fonctionnel à la plateforme. Elle orchestre notamment la création et la configuration des marketplaces, l'affectation des stores et modules, la gestion des produits, la validation des demandes, les abonnements et paiements, les partenariats avec les concessionnaires, les contrats et l'instruction des dossiers de financement. Les entités représentent l'état persistant du domaine, tandis que les services contrôlent les autorisations, les transitions d'état et la cohérence entre les agrégats. Cette séparation facilite les tests et évite de disperser les règles métier dans les contrôleurs ou dans l'interface.

Accès aux données et persistance

L'accès aux données est assuré par les repositories Spring Data JPA. Ceux-ci encapsulent les opérations de lecture et d'écriture et isolent les services métier des détails du pilote JDBC et des requêtes de persistance. Hibernate réalise la correspondance entre les entités Java et les tables PostgreSQL. Le flux logique principal est ainsi : React, contrôleurs REST, services métier, repositories JPA, puis PostgreSQL.

Sécurité et multi-tenancy

La sécurité et le contexte tenant sont des préoccupations transversales à toutes les couches du backend. Spring Security, les jetons JWT, les rôles et les contrôles applicatifs déterminent les opérations autorisées. La multi-tenancy repose sur une base et un schéma partagés : la banque constitue l'ancrage du tenant et les données sont reliées à son périmètre métier. L'isolation est donc logique, au niveau des relations et des requêtes filtrées, et non physique au moyen d'une base distincte pour chaque établissement.

Architecture physique

L'architecture physique identifie les composants mobilisés pendant l'exécution et les protocoles qui les relient. Elle ne décrit pas ici la chaîne CI/CD ni le mécanisme de conteneurisation, mais uniquement le trajet des requêtes et les dépendances nécessaires au fonctionnement de la plateforme.

Architecture physique de Matchia

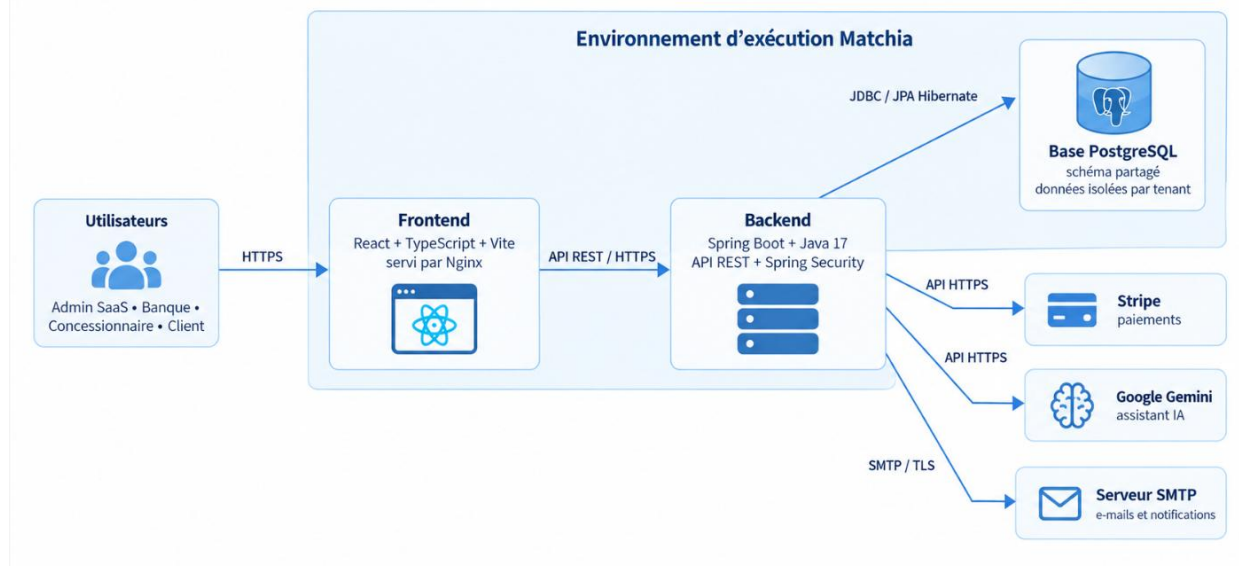


Figure 2 — Architecture physique de la plateforme Matchia

Accès utilisateur et frontend

Les utilisateurs accèdent à Matchia depuis un navigateur web. Le frontend React, compilé avec Vite et servi par Nginx dans l'environnement de déploiement présenté, fournit les ressources statiques et exécute l'interface dans le navigateur. Les échanges sont protégés par HTTPS. Le frontend ne communique ni directement avec PostgreSQL ni avec les services métier externes : il transmet les opérations applicatives au backend par l'intermédiaire de l'API REST.

Backend et base PostgreSQL

Le backend Spring Boot constitue le centre de confiance de l'application. Il authentifie les requêtes, applique les droits liés aux rôles et au tenant, exécute les règles métier et prépare les réponses retournées au frontend. L'accès à PostgreSQL s'effectue à travers JPA, Hibernate et le pilote JDBC. La base conserve notamment les utilisateurs, banques, marketplaces, stores, modules, produits, abonnements, paiements, demandes, partenariats, contrats et dossiers de financement.

Dans ce modèle multi-tenant, plusieurs banques utilisent la même infrastructure applicative et la même base relationnelle. Les services doivent donc systématiquement appliquer le contexte de banque et les restrictions de rôle avant toute consultation ou modification. Ce contrôle côté serveur constitue la garantie principale d'étanchéité entre les tenants.

Services externes

Le backend communique avec Stripe au moyen d'une API HTTPS pour préparer et confirmer les paiements associés aux abonnements. Google Gemini intervient dans les fonctions d'assistance intelligente ; l'appel est effectué par le backend afin de conserver le contrôle sur le contexte envoyé, de valider les traitements générés et d'éviter tout accès direct du modèle à PostgreSQL. Enfin, le serveur SMTP prend en charge les courriers de vérification, les notifications de traitement et les communications liées aux parcours métier. Ces dépendances restent extérieures au cœur applicatif et sont accessibles au moyen de paramètres de configuration dédiés.

Synthèse du choix architectural

L'architecture retenue répond aux besoins actuels de Matchia par un compromis cohérent entre simplicité d'exploitation et séparation des responsabilités. Le frontend autonome fournit une expérience adaptée aux différents profils, tandis que le monolithe modulaire centralise la sécurité et la cohérence transactionnelle. Les couches Controller, Service et Repository rendent le code plus lisible et testable ; PostgreSQL offre un modèle relationnel adapté aux liens entre banques, marketplaces, utilisateurs, offres et processus de financement.

Cette organisation permet également une évolution progressive. Si la charge, l'autonomie des équipes ou les contraintes réglementaires l'exigent ultérieurement, certains domaines pourront être isolés derrière des contrats plus indépendants. Une telle évolution devra toutefois être justifiée par des besoins mesurés. Dans l'état actuel du projet, l'architecture monolithique modulaire en couches demeure plus adaptée qu'une architecture microservices, car elle limite la complexité distribuée sans sacrifier la modularité fonctionnelle.